



LOGiCTECH

LogicTech

Proyecto

Hospital-VM

Presentado por
Vicente pino
Dario Donoso



LOGICTECH

Agenda

- Introducción
- Gestión: EDT y Carta Gantt
- Stack Tecnológico y Arquitectura
- Flujo del Sistema (Auth JWT)
- Demo: CRUD de Pacientes
- Persistencia y Backup de Datos
- Infraestructura y Desafíos

-



LOGICTECH

EDT - Fases del Desarrollo

N° EDT	Fase del Proyecto	Actividades Clave	Estado
1.0	Gestión	Carta del Proyecto, Monitoreo y Control.	Completado
2.0	Planificación	Estudio de factibilidad y Caso de negocio.	Completado
3.0	Análisis	Recopilación de requisitos y Seguridad.	Completado
4.0	Diseño	Diseño de alto nivel y Especificación Técnica.	En curso
5.0	Desarrollo	Construcción, Documentación e Integración.	Pendiente



LOGICTECH

Planificación y Gestión

Gestión

Carta del Proyecto y Monitoreo inicial.

Análisis

Recopilación de requisitos de seguridad.

Diseño

Especificación Técnica y Modelamiento

.

Desarrollo

Construcción y QA Hospitalario.



LOGiCTECH

Validaciones con Bean Validation



LOGICTECH

Flujo De acceso(Auth JWT)

The screenshot displays the VS Code interface with the Explorer on the left, the Git Staged Changes panel at the top, and the JwtFilter.java file open in the editor. The code implements a Spring Security filter that checks for a Bearer token in the Authorization header, extracts the username, loads the user details, and authenticates the user before allowing the request to proceed.

```
1  import java.io.IOException;
2
3  import java.servlet.FilterChain;
4  import java.servlet.ServletException;
5  import java.servlet.http.HttpServletRequest;
6  import java.servlet.http.HttpServletResponse;
7
8  import org.springframework.beans.factory.annotation.Autowired;
9  import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
10 import org.springframework.security.core.context.SecurityContextHolder;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.stereotype.Component;
13 import org.springframework.web.filter.OncePerRequestFilter;
14
15 @Component
16 public class JwtFilter extends OncePerRequestFilter {
17
18     @Autowired
19     private JwtService jwtService;
20
21     @Autowired
22     private CustomUserDetailsService userDetailsService;
23
24     @Override
25     protected void doFilterInternal(HttpServletRequest request,
26                                     HttpServletResponse response,
27                                     FilterChain chain)
28         throws ServletException, IOException {
29
30         String header = request.getHeader("Authorization");
31
32         if (header != null && header.startsWith(prefix: "Bearer ")) {
33             String token = header.substring(beginIndex: 7);
34             String username = jwtService.extractUsername(token);
35
36             UserDetails user = userDetailsService.loadUserByUsername(username);
37
38             UsernamePasswordAuthenticationToken auth =
39                 new UsernamePasswordAuthenticationToken(
40                     user, null, user.getAuthorities());
41
42             SecurityContextHolder.getContext().setAuthentication(auth);
43         }
44
45         chain.doFilter(request, response);
46     }
47 }
48
```

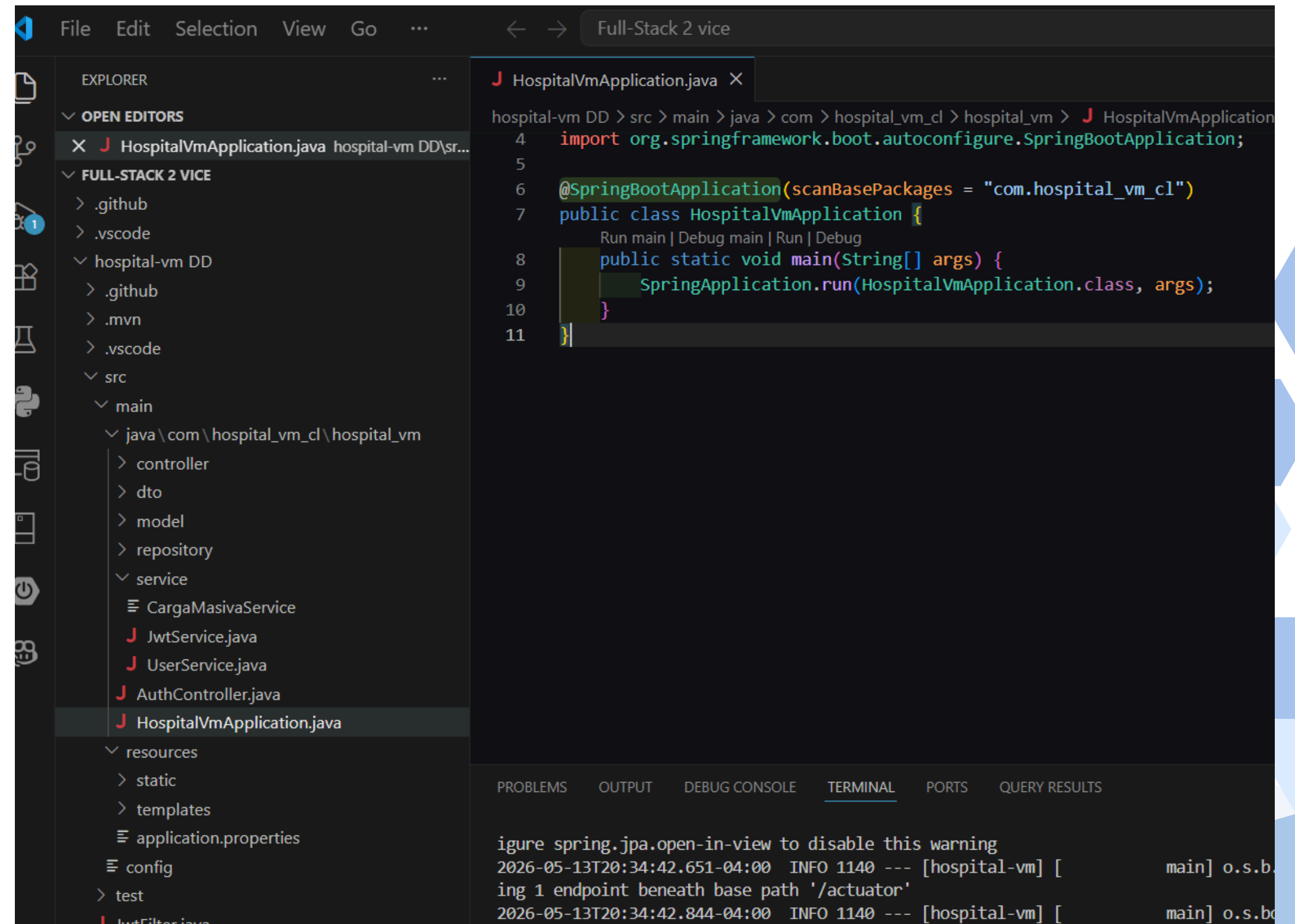
Patron CSR

Separación de Capas: Controladores, Servicios y Repositorios.

Controller: Orquesta las peticiones y respuestas REST.

Service: Centraliza la lógica de negocio del hospital.

Repository: Gestión de datos pura vía JPA



```
File Edit Selection View Go ... Full-Stack 2 vice

EXPLORER
OPEN EDITORS
X HospitalVmApplication.java hospital-vm DD\sr...
FULL-STACK 2 VICE
> .github
> .vscode
hospital-vm DD
> .github
> .mvn
> .vscode
src
main
java \com \hospital_vm_cl \hospital_vm
> controller
> dto
> model
> repository
service
CargaMasivaService
JwtService.java
UserService.java
AuthController.java
HospitalVmApplication.java
resources
static
templates
application.properties
config
test
JwtFilter.java

HospitalVmApplication.java X
hospital-vm DD > src > main > java > com > hospital_vm_cl > hospital_vm > HospitalVmApplication
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication(scanBasePackages = "com.hospital_vm_cl")
7 public class HospitalVmApplication {
8     Run main | Debug main | Run | Debug
9     public static void main(String[] args) {
10         SpringApplication.run(HospitalVmApplication.class, args);
11     }
12 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
figure spring.jpa.open-in-view to disable this warning
2026-05-13T20:34:42.651-04:00 INFO 1140 --- [hospital-vm] [main] o.s.b
ing 1 endpoint beneath base path '/actuator'
2026-05-13T20:34:42.844-04:00 INFO 1140 --- [hospital-vm] [main] o.s.b
```

Modelado y lombok

```
hospitalVmApplication.class  CargaMasivaService ...\classes\...  PacienteRepository.class  UserRepository

1  // Source code is unavailable, and was generated by the Fernflower decompiler.
2  package com.hospital_vm_cl.hospital_vm.model;
3
4  import jakarta.persistence.Column;
5  import jakarta.persistence.Entity;
6  import jakarta.persistence.GeneratedValue;
7  import jakarta.persistence.GenerationType;
8  import jakarta.persistence.Id;
9  import jakarta.persistence.Table;
10 import java.util.Date;
11 import lombok.Generated;
12
13 @Entity
14 @Table(
15     name = "paciente"
16 )
17 public class Paciente {
18     @Id
19     @GeneratedValue(
20         strategy = GenerationType.IDENTITY
21     )
22     private Integer id;
23     @Column(
24         unique = true,
25         length = 13,
26         nullable = false
27     )
28     private String run;
29     @Column(
30         nullable = false
31     )
32     private String nombres;
33     @Column(
34         nullable = false
35     )
36     private String apellidos;
37     @Column(
```

Limpieza y Eficiencia

Utilizamos Lombok para reducir el código repetitivo (Boilerplate).

- **@Data:** Getters/Setters automáticos.
- **@Entity:** Mapeo directo a tabla MySQL.
- **@NoArgsConstructor:** Constructor vacío para JPA.

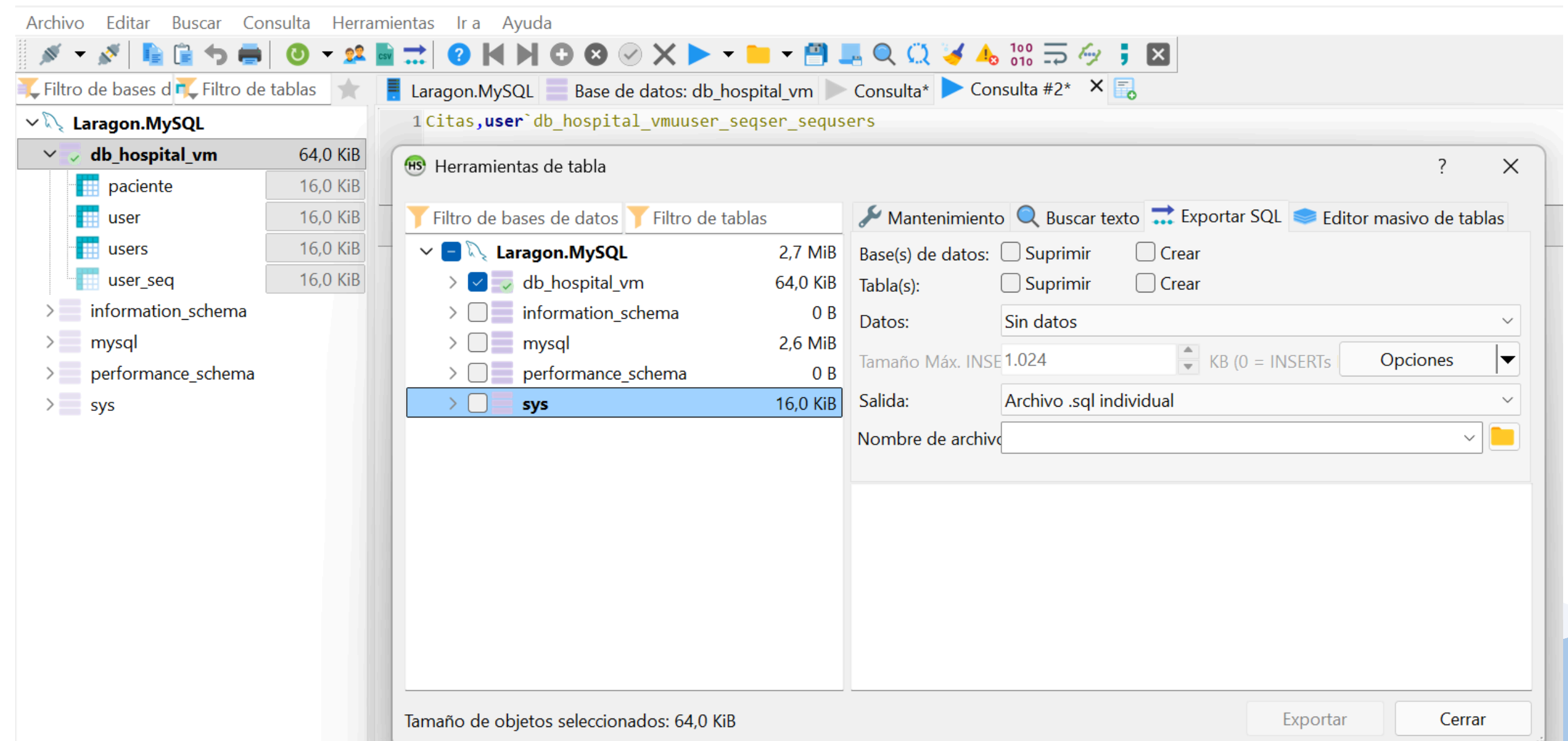


LOGICTECH

Persistencia y Backend

Implementación real con Laragon y Hibernate

Base de Datos: MySQL



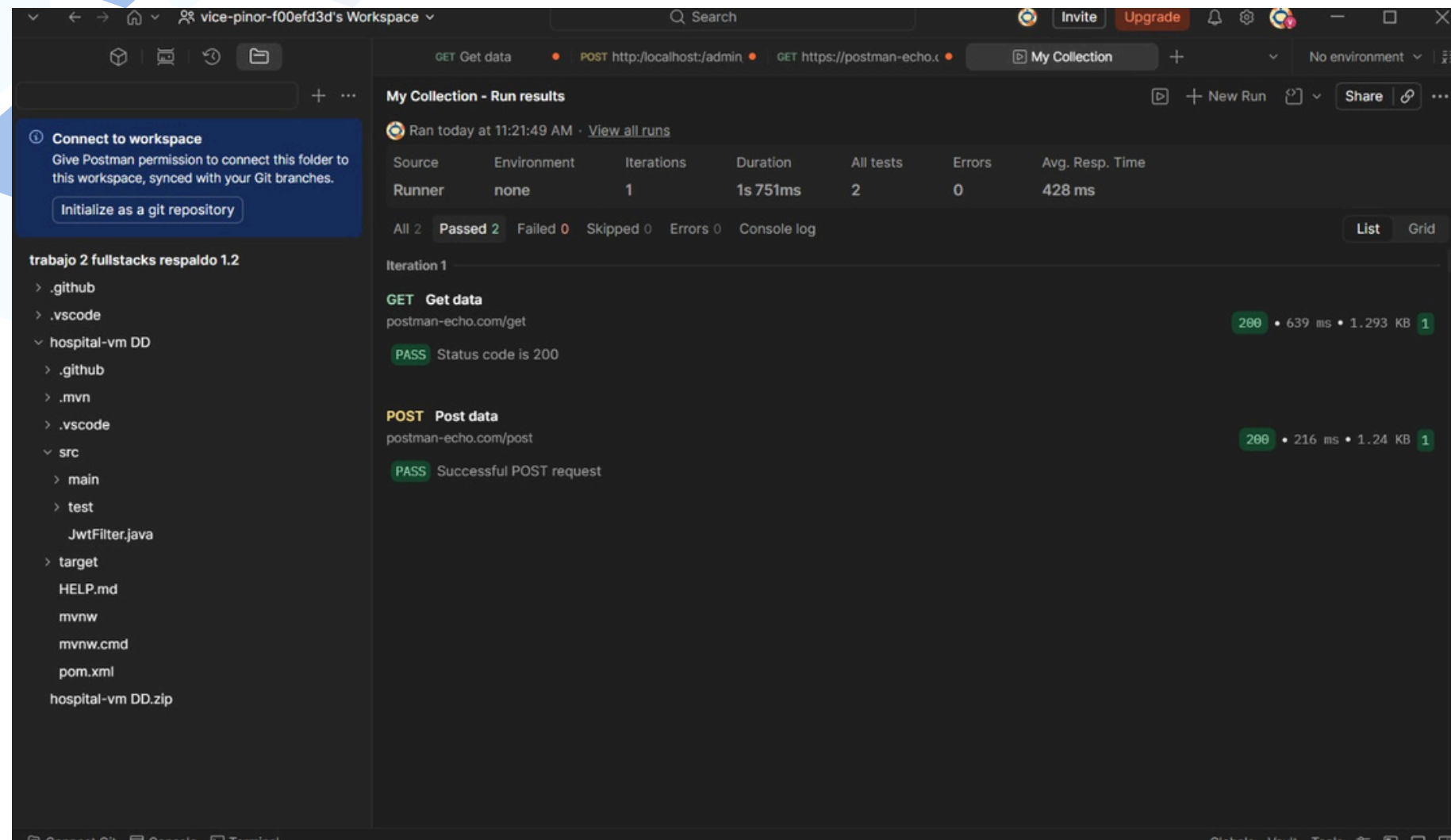


LOGICTECH

Seguridad: Auth JWT

Flujo de Autorización

- 1.Registro: Creación de usuario en DB.
- 2.Login: Servidor valida y emite JWT.
- 3.Auth: El cliente incluye el token en el Header.
- 4.Acceso: Endpoint CRUD autorizado.





LOGICTECH

Persistencia y Continuidad

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/api/pacientes`
- Method:** `GET`
- Auth Type:** `Bearer Token`
- Token:** `token-DDev`
- Response Status:** `200 OK` (145 ms, 3.16 KB)
- Response Body (JSON):**

```
[
  {
    "id": 1,
    "run": "93113348-9",
    "nombres": "Elias",
    "apellidos": "Torres",
    "fechaNacimiento": "1993-01-01T03:00:00.000Z",
    "correo": "eli.torres@example.com"
  },
  {
    "id": 2,
    "run": "98765432-1",
    "nombres": "Señor",
    "apellidos": "Lápiz",
    "fechaNacimiento": "1990-02-02T03:00:00.000Z",
    "correo": "senor.lapiz@example.com"
  },
  {
    "id": 3,
    "run": "87654321-2",
    "nombres": "Señor",
    "apellidos": "Lápiz",
    "fechaNacimiento": "1990-02-02T03:00:00.000Z",
    "correo": "senor.lapiz@example.com"
  }
]
```



LOGICTECH

Respuestas Controladas

Escenario	Código HTTP	Componente
Paciente no encontrado	404 Not Found	@ControllerAdvice
Error en validación de datos	400 Bad Request	Bean Validation
Acceso no autorizado	401 Unauthorized	JWT Security Filter
Éxito en creación/update	201 Created / 200 OK	ResponseEntity



LOGICTECH

Demostración CRUD

```
hospital-vm DD > pom.xml 1 X J HospitalVmApplicationTests.class
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5   <parent>
7     <artifactId>spring-boot-starter-parent</artifactId>
8     <version>4.0.6</version>
9     <relativePath/> <!-- lookup parent from repository -->
10  </parent>
11  <groupId>com.hospital-vm-cl</groupId>
12  <artifactId>hospital-vm</artifactId>
13  <version>0.0.1-SNAPSHOT</version>
14  <name/>
15  <description/>
16  <url/>
17  <licenses>
18    <license/>
19  </licenses>
20  <developers>
21    <developer/>
22  </developers>
23  <scm>
24    <connection/>
25    <developerConnection/>
26    <tag/>
27    <url/>
28  </scm>
29  <properties>
30    <java.version>21</java.version>
31  </properties>
32  <dependencies> Add Spring Boot Starters...
33
34    <dependency>
35      <groupId>org.springframework.boot</groupId>
36      <artifactId>spring-boot-starter-actuator</artifactId>
37    </dependency>
38
39    <dependency>
40      <groupId>org.springframework.boot</groupId>
41      <artifactId>spring-boot-starter-data-jpa</artifactId>
42    </dependency>
43    <dependency>
44      <groupId>org.springframework.boot</groupId>
```